TECHNICAL PAPER | EDUCATIONAL & AUTHORIZED-TESTING USE ONLY | V1 · SEPTEMBER 2026

# Trust Me, I'm a Camera

*A man-in-the-middle and RTP replay attack against RTSP video-surveillance streams*

Martín López Aranburu • linkedin.com/in/martín-lópez-aranburu • Barcelona Cybersecurity Congress, November 2026

**ABSTRACT**

RTSPoofing is a working man-in-the-middle tool against the RTSP/RTP pipeline that most consumer and small-business IP cameras and network video recorders (NVRs) rely on. By combining ARP-based positioning with a byte-transparent RTSP relay and RTP-level frame substitution, an on-path attacker can quietly replace a live camera feed with pre-recorded footage. The recording NVR keeps logging a stream that looks entirely valid: the sequence numbers, timestamps and sender identity all line up, and the RTCP wall-clock can be forged as well. This paper covers the three-stage architecture, the extension to UDP-transported RTP, and the defenses that close each stage.

**KEYWORDS** RTSP · RTP/RTCP · man-in-the-middle · ARP spoofing · video surveillance · IoT/embedded security · protocol authentication

# Contents


**1 Introduction** **3**

**2 Background: RTSP and RTP** **3**
2.1 The RTP header . . . . . 3

**3 Threat model and architecture** **4**

**4 Phase I: network-layer positioning** **5**

**5 Phase II: transparent interception** **5**
5.1 Byte-transparent RTSP relay, credentials included . . . . . 5
5.2 RTPProbe: live parameter detection . . . . . 5

**6 Phase III: media substitution** **6**
6.1 Continuity rewrite . . . . . 6
6.2 Fixed-rate pacing . . . . . 6
6.3 Two independent notions of “now” . . . . . 6

**7 Extending to UDP transport** **7**

**8 Empirical scope** **7**

**9 Detection and countermeasures** **8**

**10 Discussion: assumptions and limitations** **8**

**11 Conclusion** **9**

**12 Ethics and responsible disclosure** **9**

# 1 Introduction

Most physical-security equipment now runs over ordinary IP networks, and those networks were never built to check who is actually sending traffic on them.

Most IP cameras and the NVRs that record them speak RTSP in plaintext. The RTP session they negotiate has no integrity protection, so each side trusts whatever arrives on the agreed channel. This paper works through a practical question: can an NVR be made to record a fabricated feed with no firmware changes and no credentials, and where does that approach finally break down?

That question is answered by **RTSPoofing**, an asynchronous Rust/Tokio proxy of roughly 1,600 lines across eight modules that runs the full attack against both transports RTSP supports. Sections 4–7 follow its three stages and the UDP extension in execution order. Sections 8 and 10 set out what was and was not verified, and Section 9 pairs each stage with a defense that is cheap to deploy.

> **SCOPE**
> All testing described here was performed in isolated lab environments or under explicit written authorization. Findings labeled *informal* come from my own test environments and are not a controlled study across a representative device population; see §8 and §10 for the caveats.

# 2 Background: RTSP and RTP

RTSP (Real Time Streaming Protocol, RFC 2326) is the out-of-band control channel almost every IP camera speaks. A client, which here is the NVR, sends text requests to set up and control a media session.

```
NVR -> CAM  DESCRIBE rtsp://192.168.1.10/stream RTSP/1.0
CAM -> NVR  200 OK -- SDP body: available media, codecs, transport options

NVR -> CAM  SETUP rtsp://192.168.1.10/stream/track1 RTSP/1.0
              Transport: RTP/AVP;unicast;client_port=8000-8001
CAM -> NVR  200 OK -- Transport: ...;server_port=9000-9001  (or interleaved=0-1)

NVR -> CAM  PLAY rtsp://192.168.1.10/stream RTSP/1.0
CAM -> NVR  200 OK -- media now flowing
```

The `SETUP` exchange matters most here, since it picks one of two transports for the RTP/RTCP that follows. **TCP-interleaved** carries the media inside the same RTSP connection, each frame prefixed with a `$` marker, a channel number and a 2-byte length (RFC 2326 §10.4). **UDP** sends RTP and RTCP as separate datagrams on the `client_port/server_port` pairs agreed during `SETUP`. Both are common; §7 explains why UDP needs separate handling.

## 2.1 The RTP header

Once negotiated, media flows as a sequence of RTP packets (RFC 3550). Three header fields carry the entire state a receiver uses to make sense of the stream:

```
 0                   1                   2                   3
 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|V=2|P|X|  CC   |M|     PT      |       Sequence Number         |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|                           Timestamp                           |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|           Synchronization Source (SSRC) identifier            |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
```

| | |
|---|---|
| **Sequence Number** | 16-bit, increments per packet. RTP runs over lossy UDP by design, so conforming receivers conceal isolated gaps rather than terminating the session (RFC 3550 §8). A large or backward discontinuity is what actually surfaces as a defect. |
| **Timestamp** | 32-bit media clock, conventionally 90 kHz for video regardless of frame rate. Should advance monotonically; a stall or reversal is what a jitter buffer notices. |
| **SSRC** | 32-bit sender identity. A receiver is expected to treat a change as a new source and reset decoder state (RFC 3550 §8.2) — disruptive to conceal, not fatal to avoid. |

None of these fields carries a signature or MAC. SRTP (RFC 3711) would add one, but it is practically absent from consumer and SMB CCTV. Those three unauthenticated fields are the whole surface the rest of this paper works with.

## 3 Threat model and architecture

The attacker shares a broadcast domain (an unsegmented VLAN) with the camera and the NVR, which is the common case in the deployments described in §8, and holds no credentials for either device. The attacker is not after the stream itself; the aim is to replace its *content* while the NVR keeps recording what it takes to be the live camera.

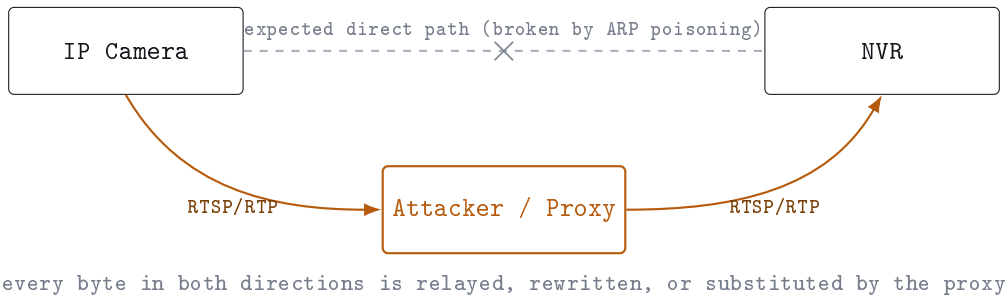



**Figure 1:** ARP spoofing does not open a new path. It breaks the one the camera and NVR think they are using, so both sides' traffic is redirected through the attacker.

The implementation executes this as three stages, one `MITMSession` task per accepted TCP connection:

1. **Phase I, network-layer positioning** (§4): ARP spoofing places the attacker on-path at layer 2.
2. **Phase II, transparent interception** (§5): a byte-level RTSP/RTP relay with live parameter auto-detection.
3. **Phase III, media substitution** (§6): paced, continuity-preserving RTP frame injection.

Both transports (TCP-interleaved and UDP, §7) run through the same frame-handling function, `process_camera_frame()`, so that probing, capture and injection cannot quietly behave differently between them.

## 4 Phase I: network-layer positioning

Camera and NVR normally address each other directly. The attacker poisons both ARP caches with periodic forged replies, typically using a tool such as `arpspoof`, so each side resolves the other's IP to the attacker's MAC:

```
arpspoof -i eth0 -t <NVR> <CAM>
arpspoof -i eth0 -t <CAM> <NVR>
```

With IP forwarding enabled on the attacker host, traffic still flows end to end and neither link appears to drop; it just routes through the attacker first. The technique is old and well documented. All it needs is layer-2 adjacency, which is normal for a camera and NVR sharing a flat, unsegmented network (§8).

> **WHAT STOPS IT**
> Dynamic ARP Inspection (DAI) validates ARP packets against a DHCP-snooping or static binding table and drops ones that don't match, which is a complete stop for this phase.

## 5 Phase II: transparent interception

With traffic now passing through it, the proxy does two things at once: it relays the RTSP control channel byte for byte, and it quietly characterises the RTP media it is passing on.

### 5.1 Byte-transparent RTSP relay, credentials included

An `RtspTextScanner` collects raw TCP bytes and emits complete RTSP messages as they arrive. A message ends at the blank line that closes the header block, plus any `Content-Length` body; on the media side an `InterleavedParser` finds complete `$`-framed RTP frames the same way. Every request and response is forwarded unchanged, apart from the Transport-header rewrite a UDP `SETUP` needs (§7).

That includes any `Authorization` header. RTSP supports Basic and Digest authentication on these requests, and plenty of real deployments turn it on, yet the proxy never needs the camera's password. The NVR computes its own credentials as usual and sends them over a connection it thinks is direct, and the proxy relays that header like any other byte. Device authentication keeps an unauthorized new client from opening a session. It does nothing about an on-path relay sitting inside a session between two parties that have already authenticated.

### 5.2 RTPProbe: live parameter detection

The operator does not have to supply the payload type, SSRC or frame rate. The proxy reads them off the live stream during a short warm-up window. Each RTP frame casts a vote for payload type, SSRC and, in the interleaved case, channel number, and whichever value leads when the window closes (set by both a duration and a minimum frame count) is taken as correct. It is not just the first packet, because the frames right after `PLAY` are often noisy.

```
[00:00.0] Analyzing stream...
[00:01.2] PT=96 seen 847 times -> dynamic H264
[00:02.1] SSRC=0xDEAD1234 dominant (99.8% of frames)
[00:02.8] Estimated FPS: 25.0 Hz (via RTP timestamp deltas)
[00:03.0] Codec: H264 (fragmented FU-A detected)
[00:03.0] Probe complete -- starting injection
```

Frame rate comes with no out-of-band signalling at all. It rests on one fact: RTP's video clock ticks at a fixed 90,000 Hz whatever the real frame rate. Taking the median of the RTP timestamp deltas between consecutive packets over a sliding window (the last 60 valid samples, dropping deltas that wrap, exceed one clock period, or come with an implausibly small wall-clock gap) settles on

```
fps = 90000 / dts

e.g.  3,600 ticks between two frames -> 90000 / 3600 = 25 fps
```

# 6 Phase III: media substitution

Once the parameters are known, an injector swaps the live stream for frames read from a `.rtp` capture file. That file is either recorded earlier from the same camera in the proxy's own `record` mode (an 8-byte record header and the raw RTP payload per frame, written on a separate blocking task so a slow disk never stalls the live relay) or built from any source video with `ffmpeg`.

## 6.1 Continuity rewrite

Each injected frame has its sequence number and timestamp rewritten to carry on from the last values seen on the *live* stream when injection begins, and its SSRC set to the camera's real one (or an operator override). At the RTP-metadata level the switch from live to injected leaves nothing to catch: no gap in the numbering, no backward jump, and no change of source.

## 6.2 Fixed-rate pacing

Frames go out on a `tokio::time::interval` that ticks once every `1/fps` seconds, set to `MissedTickBehavior::Delay` so a slow write or a scheduler hiccup pushes the next tick back instead of firing a burst to catch up. An injector that just sends as fast as the network allows is the easiest way to get caught: the dropped or corrupted frames and the bursty arrival timing stand out even to a monitor that never decodes a single frame.

## 6.3 Two independent notions of "now"

A camera usually carries two timestamps. One is the on-screen display burned into the pixels by the camera firmware. The other is the NTP field in an RTCP Sender Report, a wall-clock value that some VMS and forensic export tools read on their own, without looking at the picture. The first is free to spoof, since an injected clip already carries whatever OSD time it was recorded with. For the second, the proxy sends its own SR every `rtcp_sr_interval` seconds, stamped with the real current time and counting the injected stream's own packets and octets:

| Bytes | Field | Content |
|---|---|---|
| 0–4 | `Header` | V=2, P=0, RC=0 · PT=200 (SR) · length=6 words |
| 4–8 | `SSRC` | Sender identifier — matches the injected stream's SSRC |
| 8–12 | `NTP seconds` | Wall-clock "now" at send time |
| 12–16 | `NTP fraction` | Sub-second wall-clock precision |
| 16–20 | `RTP timestamp` | Media-clock value at this instant, same clock as the stream |
| 20–24 | `Packet count` | Running total for this synthetic sender |
| 24–28 | `Octet count` | Running payload byte total |

What matters more than the RTCP mechanics is the priority. Spoofing the picture already beats the audience that counts, a person reviewing footage. Forging the RTCP time only helps against tools that read the wall clock rather than the frame, and it is here for completeness against that narrower, more technical case, not because it is the main exposure.

## 7 Extending to UDP transport

Sections 5–6 assumed TCP-interleaved media. `SETUP` just as often negotiates plain UDP, with RTP and RTCP as separate datagrams on their own ports, and that case is easy to get wrong. ARP spoofing puts the attacker on-path at *layer 2*, but once IP forwarding is on, a UDP datagram sent to the camera's or NVR's real IP is forwarded by the kernel at the IP layer and never reaches the proxy's sockets. Sitting on-path at L2 is not enough at the application layer. To inject into UDP media the proxy has to own the datagrams, and that means acting where the transport gets negotiated, in the `SETUP` exchange.

The fix is to rewrite that negotiation, one layer up from the ARP spoofing below it:

1. The NVR's `SETUP` declares `client_port=8000-8001`; the proxy substitutes ports it owns before forwarding the request to the camera.
2. The camera, none the wiser, now sends its RTP/RTCP straight to the proxy.
3. The camera's response declares `server_port=9000-9001`; the proxy substitutes ports it owns there too, and restores the NVR's own original `client_port` in the same response so the NVR isn't confused by a value it never sent.
4. The NVR, none the wiser, now sends its RTCP receiver reports straight to the proxy as well.

Why four sockets and not two: some camera and NVR implementations check that incoming RTP/RTCP really comes from the port pair declared during `SETUP`, so media has to be sent from the socket each side was told about, not just any socket that can reach it. That is a conservative design choice. It was not tested against a catalogue of specific firmware, a limitation §10 repeats.

All four sockets feed the same `process_camera_frame()` path as the TCP-interleaved case, so probing, capture, injection and RTCP forgery stay identical across the two transports. Autodetection, pacing and the forged timestamps behave the same whichever transport a camera uses.

## 8 Empirical scope

Across my own test environments:

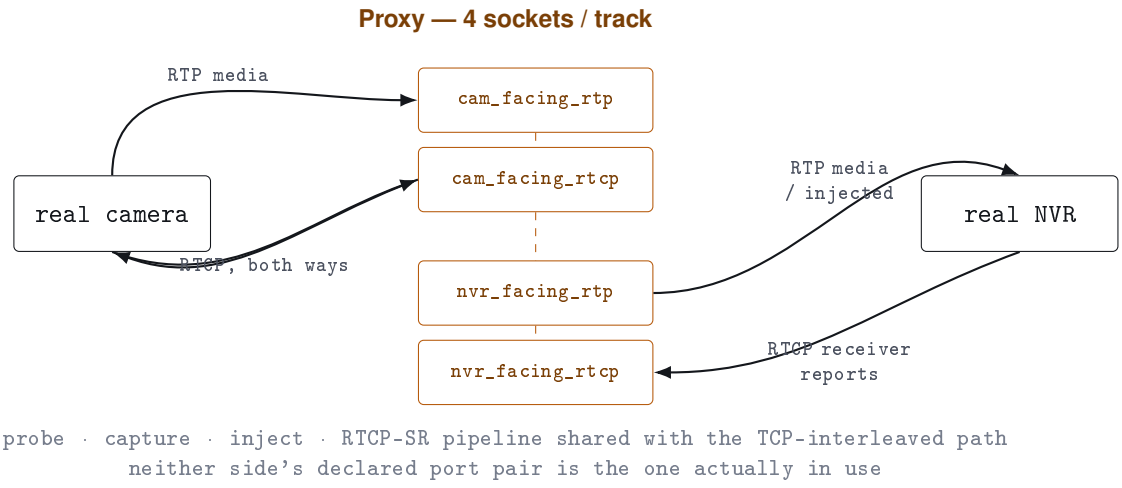



**Figure 2:** Four sockets, not two: each side's real port pair is rewritten to one the proxy owns, so both media *and* RTCP reports terminate at the proxy in both directions.

| Environment | Result | Condition |
|---|---|---|
| Home networks | **Reliably works** | Consumer router, no DAI |
| Small business | **Usually works** | Unmanaged switch, no hardening |
| Dedicated CCTV/NVR | **Often works** | Flat network, consumer-grade gear |

**CAVEAT, STATED PLAINLY**

These are informal observations across the environments I tested, not measurements from a controlled study across a representative device population. Treat category boundaries as directional, not statistical.

## 9 Detection and countermeasures

| Stage | Defense | Why it works |
|---|---|---|
| Phase I | **Dedicated VLAN** for camera traffic | Removes the flat-network adjacency §4 and §8 depend on. |
| Phase I | **Dynamic ARP Inspection** | Managed switches validating ARP against a DHCP-snooping or static binding table stop the spoof in §4 outright. |
| Phase II–III | **RTSP over TLS** (RTSPS) | Removes the proxy's ability to read or rewrite the plaintext control channel that §5–§7 all depend on — the single defense that breaks the whole chain at once. |
| All stages | **ARP / network anomaly monitoring** | Detective control (arpwatch, NDR/XDR) for cases the preventive controls above miss or are misconfigured. |

## 10 Discussion: assumptions and limitations

- **Transport mix.** UDP is common in the field but not universal; some deployments default to TCP-interleaved to keep UDP loss from showing up as visible glitches. TCP-only coverage is not enough in general, and the claim that UDP dominates is my own observation rather

than a survey.

- **Receiver loss tolerance favors the attacker.** Because RTP receivers hide isolated gaps by design (§2), the continuity work in §6.1 is about avoiding a *visible* break, not about keeping the session alive; it would survive far rougher handling. The margin is wider than a literal reading of "must stay continuous" makes it sound.
- **Four-socket rationale is a design assumption, not a confirmed finding.** §7 states the source-port-validation rationale as the reason for the design; which specific NVR/camera implementations actually enforce it remains an open question for anyone extending this work.
- **RTCP-SR consumption by VMS/forensic tooling** (§6.3) is plausible and consistent with the field's documented purpose (RFC 3550 §6.4.1), but was not confirmed against a specific named product.

## 11 Conclusion

- IP cameras and NVRs trust anything that speaks TCP to them on the right port.
- RTSP/RTP carries no built-in stream authentication; SRTP exists and is essentially absent from this market segment.
- The attack works without touching camera or NVR firmware, and without camera credentials.
- Detection from the NVR's own vantage point alone is close to impossible without independent network-level monitoring.
- Most of the home and SMB installs surveyed were vulnerable as shipped, with default configuration.

*If a security system can be defeated by a few thousand lines of an evening's work, the sticker on the box was never the whole story.*

## 12 Ethics and responsible disclosure

All testing described in this paper was conducted in isolated lab environments or under explicit, written authorization from the network owner. The implementation is released for research and authorized security testing only. Testing a network without the owner's explicit authorization is illegal in most jurisdictions and is not endorsed by this work in any form; I bear no responsibility for uses inconsistent with this statement.